



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:**

*Разработка программно-аппаратного комплекса
поддержки мобильного приложения визуализации
результатов работы модификаций эволюционного
алгоритма оптимизации*

Студент ИУ9-82Б
(Группа)

(Подпись, дата)

А.В. Волохов
(И.О. Фамилия)

Руководитель НИР

(Подпись, дата)

Д.П. Посевин
(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ОПИСАНИЕ БАЗЫ ПРАКТИКИ	3
ВВЕДЕНИЕ	4
1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Задача глобальной оптимизации и метаэвристики	6
1.2 Алгоритм биогеографической оптимизации	7
1.3 Модификации модели миграции биогеографической оптимизации	8
1.4 Модификации базового алгоритма биогеографической оптимизации	10
1.5 Метод роя частиц и сравнение с методом биогеографической оптимизации	11
2 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ	13
2.1 Требования к визуализации в реальном времени	13
2.2 График сходимости	14
2.3 Визуализация пространства поиска	15
2.4 Трёхмерный поверхностный график	16
2.5 Анимированное представление динамики	18
3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	20
3.1 Протоколы передачи данных и обоснование выбора WebSocket	20
3.2 Серверная часть: Python	21
3.3 Формат обмена данными: JSON	23
3.4 Клиентская часть: Dart и Flutter	24
ЗАКЛЮЧЕНИЕ	26
СПИСОК ЛИТЕРАТУРЫ	28

ОПИСАНИЕ БАЗЫ ПРАКТИКИ

Местом прохождения практики является Отдел информационных технологий и поддержки (ОИТП) МГТУ им. Н.Э. Баумана — структурное подразделение, обеспечивающее работу информационно-технологической инфраструктуры всего университетского комплекса. В зону ответственности отдела входит широкий круг задач: техническая поддержка пользователей, администрирование корпоративной сети и серверного оборудования, а также поддержка и развитие информационных систем университета.

Деятельность ОИТП охватывает как повседневную эксплуатацию цифровых сервисов — систем аутентификации, учебных порталов, систем документооборота, — так и задачи по внедрению новых программных решений в интересах образовательного и административного процессов. Специалисты отдела обеспечивают бесперебойный доступ к инфраструктуре для преподавателей, студентов и сотрудников университета.

ВВЕДЕНИЕ

Задачи глобальной оптимизации возникают в самых разных областях: при настройке гиперпараметров моделей машинного обучения, проектировании технических систем, решении инженерных и научных задач. На практике целевая функция нередко имеет сложный рельеф с большим числом локальных минимумов, что делает применение классических градиентных методов малоэффективным. В таких случаях широко используются популяционные метаэвристические алгоритмы, которые исследуют пространство решений без вычисления производных и способны избегать локальных ловушек за счёт коллективного поиска.

Одним из таких методов является алгоритм биогеографической оптимизации (Biogeography-Based Optimization, далее — ВВО), предложенный Д. Саймоном в 2008 году. В основе алгоритма лежит математическая модель миграции видов между островами. ВВО обладает рядом привлекательных свойств: простой параметрической структурой, гибкостью в части выбора модели миграции, а также возможностью модификации различными механизмами адаптации. В рамках настоящей работы рассматриваются несколько вариантов алгоритма: классический ВВО, варианты с синусоидальной и смешанной моделями миграционных кривых, а также модифицированные версии каждого из них, дополненные адаптивной мутацией, детектором стагнации с частичным рестартом и локальным поиском.

Наряду с алгоритмическими вопросами актуальной является задача визуального анализа работы методов оптимизации. Наблюдение за поведением популяции в процессе поиска — графиками сходимости, положением особей в пространстве решений — позволяет лучше понять характер работы алгоритма, выявить проблемы преждевременной сходимости или стагнации. Особый интерес представляет отображение таких данных в реальном времени на мобильном устройстве, что обеспечивает доступность инструмента исследователю без привязки к стационарному рабочему месту.

Для реализации подобной системы необходима согласованная работа трёх компонентов: вычислительного сервера, выполняющего оптимизацию, транспортного уровня, обеспечивающего потоковую передачу данных, и мобильного клиента, осуществляющего визуализацию. Это порождает ряд вопросов: какой протокол связи наиболее подходит для передачи прогресса итерация за итерацией, как орга-

низовать очередь задач на сервере, какими средствами реализовать графическое представление данных в мобильном приложении.

Целью данной научно-исследовательской работы является теоретическое исследование, охватывающее три направления:

- теоретические основы и математические модели алгоритма BBO и его модификаций, а также сравнительный анализ с методом роя частиц (Particle Swarm Optimization, далее — PSO);
- методы потоковой визуализации результатов эволюционной оптимизации на мобильном устройстве в реальном времени;
- программный стек для реализации информационной системы, включающей серверную часть на Python, протокол обмена данными на основе WebSocket и мобильный клиент на Dart и Flutter.

Результаты проведённого исследования служат теоретической основой для дальнейшей практической разработки программно-аппаратного комплекса.

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Задача глобальной оптимизации и метаэвристики

Задача глобальной оптимизации в общем виде формулируется как поиск глобального минимума целевой функции

$$f(x) \rightarrow \min, \quad x \in D \subset \mathbb{R}^d, \quad (1)$$

где D — допустимая область поиска, d — размерность пространства решений. На практике целевая функция нередко является многоэкстремальной: она имеет множество локальных минимумов, и задача состоит в том, чтобы найти именно глобальный. Классические градиентные методы в такой ситуации малоэффективны: они сходятся к ближайшему локальному минимуму и не располагают механизмами выхода из него.

Для решения многоэкстремальных задач применяются метаэвристические алгоритмы. Это класс итеративных методов, инспирированных процессами живой природы: эволюцией, поведением стай животных, миграцией видов. Главная отличительная черта метаэвристик — работа с целой популяцией потенциальных решений одновременно. Это позволяет охватывать сразу несколько областей пространства поиска и снижает вероятность застрять в одном локальном минимуме. Случайные механизмы — мутации, обмен информацией между особями, отбор — дополнительно обеспечивают разнообразие популяции и препятствуют её преждевременной сходимости [1].

Метаэвристики не требуют вычисления производных и не накладывают жёстких условий гладкости или выпуклости на целевую функцию. Это делает их универсальным инструментом для задач, где аналитическое исследование функции затруднено. К наиболее распространённым алгоритмам этого класса относятся: генетические алгоритмы [3], PSO, алгоритмы муравьиной колонии, а также ВВО [2].

Для объективного сравнения алгоритмов между собой принято использовать наборы тестовых функций с заранее известными глобальными минимумами.

Тестовые функции подбираются так, чтобы демонстрировать разный характер рельефа. Унимодальная функция сферы — простейший случай без локальных ловушек — позволяет оценить скорость сходимости алгоритма. Функция Растргина является сильно мультимодальной: она содержит большое число регулярно расположенных локальных минимумов, что создаёт серьёзные трудности для большинства методов. Функция Экли обладает одним глобальным минимумом в окружении почти плоской области с осцилляциями, что проверяет способность метода точно его локализовать. Функция Букина N.6 имеет узкий криволинейный «овраг», вдоль которого расположен минимум, — это проверяет способность алгоритма двигаться вдоль сложных структур в пространстве решений. Использование подобного набора позволяет всесторонне оценить поведение метода на задачах разной структуры и сложности.

1.2 Алгоритм биогеографической оптимизации

Алгоритм биогеографической оптимизации был предложен Д. Саймоном в 2008 году [6]. Его идея опирается на математическую модель биогеографии, описывающую распределение видов между островами. В терминах оптимизации каждое решение-кандидат рассматривается как отдельный остров, а качество решения интерпретируется как индекс пригодности среды обитания (Habitat Suitability Index, далее — HSI). Хорошим решениям соответствуют острова с высоким HSI, плохим — с низким.

Основными механизмами алгоритма являются миграция и мутация. Пусть имеется популяция из N решений $X_1, \dots, X_N$, отсортированных по возрастанию значения целевой функции для задачи минимизации. Тогда ранг $r = 1$ получает наихудшее решение, а ранг $r = N$ — наилучшее. Каждому решению сопоставляются две величины: скорость эмиграции λ_r и скорость иммиграции μ_r . В линейной модели Саймона они определяются как

$$\lambda_r = \frac{r}{N + 1}, \quad \mu_r = 1 - \lambda_r, \quad r = 1, \dots, N. \quad (2)$$

Смысл данных формул прост: хорошее решение с высоким рангом имеет высокую скорость эмиграции и низкую скорость иммиграции, то есть является

донором информации. Плохое решение, напротив, активно принимает информацию от доноров [7].

Миграция реализуется по схеме частичной иммиграции. Для каждого решения-получателя X_k , где $k > e$, а e — число элитных особей, каждая компонента s вектора X_k независимо обновляется с вероятностью μ_k : если компонента выбрана для иммиграции, она заменяется соответствующей компонентой решения-донора, выбранного случайно из популяции пропорционально скоростям эмиграции $\{\lambda_r\}$.

Механизм мутации вносит случайные изменения в популяцию. Для каждой компоненты s решения X_k с вероятностью p_{mut} выполняется замена: компонента получает новое значение, равномерно распределённое в допустимых границах. Мутация поддерживает разнообразие популяции и препятствует её вырождению в окрестности одного минимума.

Элитарность означает, что e лучших решений текущего поколения переносятся в следующее без изменений, защищая лучший найденный результат от случайного ухудшения в ходе миграции и мутации.

- Общая схема работы классического ВВО в одном поколении выглядит так:
- вычислить значения целевой функции для всей популяции и отсортировать решения по качеству;
 - по рангу каждого решения вычислить λ_r и μ_r ;
 - скопировать e лучших решений в новую популяцию без изменений;
 - для каждого из оставшихся решений выполнить частичную иммиграцию: пройти по всем компонентам и с вероятностью μ_k заменить компоненту на значение из случайного донора;
 - применить мутацию: для каждой компоненты с вероятностью p_{mut} заменить её случайным значением из допустимой области;
 - обновить лучшее найденное решение и историю сходимости.

1.3 Модификации модели миграции биогеографической оптимизации

В классическом ВВО скорости миграции описываются линейными функциями от ранга. Форма кривой миграции напрямую влияет на то, насколько интенсив-

но лучшие решения передают свои характеристики худшим и как распределён обмен информацией по всей популяции. Линейная модель равномерно распределяет эту интенсивность, что в ряде задач приводит к преждевременной сходимости: все особи быстро становятся похожими друг на друга, и поиск теряет разнообразие.

Для решения этой проблемы была предложена синусоидальная модель миграции [8]. В ней кривые эмиграции и иммиграции задаются формулами

$$\lambda_r^{\sin} = \frac{1 - \cos\left(\pi \frac{r}{N}\right)}{2}, \quad \mu_r^{\sin} = 1 - \lambda_r^{\sin}, \quad r = 1, \dots, N. \quad (3)$$

Синусоидальная кривая нелинейна: в середине ранговой шкалы интенсивность обмена изменяется медленнее, чем на краях. Это означает, что решения среднего качества меньше подвержены давлению иммиграции, что позволяет им дольше сохранять своеобразие и исследовать свои окрестности. На краях же — у очень хороших и очень плохих особей — обмен происходит интенсивнее, что соответствует естественной биогеографической логике.

Другой подход — смешанная модель — строит итоговые скорости миграции как линейную комбинацию линейной и синусоидальной моделей [6]:

$$\lambda_r^{\text{mix}} = \alpha \lambda_r^{\text{lin}} + (1 - \alpha) \lambda_r^{\sin}, \quad \mu_r^{\text{mix}} = 1 - \lambda_r^{\text{mix}}, \quad r = 1, \dots, N, \quad (4)$$

где $\alpha \in [0, 1]$ — параметр смешения, определяющий долю линейной составляющей. При $\alpha = 1$ получается чисто линейная модель, при $\alpha = 0$ — чисто синусоидальная. Выбор промежуточного значения α позволяет гибко настраивать форму кривой и баланс между исследованием пространства и уточнением найденных решений.

Все три варианта — линейный, синусоидальный и смешанный — используют одну и ту же схему иммиграции и мутации. Разница состоит исключительно в том, как вычисляются λ_r и μ_r . Это делает модель миграции независимым и легко заменяемым компонентом алгоритма.

1.4 Модификации базового алгоритма биогеографической оптимизации

Помимо изменения формы кривых миграции, качество поиска можно существенно улучшить, дополнив базовый алгоритм механизмами адаптации. Все три рассматриваемые ниже модификации применимы к любому из вариантов ВВО и не зависят от конкретной формы кривых миграции.

Первая модификация — адаптивная мутация. В базовом ВВО вероятность мутации p_{mut} фиксирована на протяжении всей работы алгоритма. Между тем в начале поиска высокая мутация полезна: она обеспечивает разнообразие популяции и позволяет охватить широкую область пространства решений. В конце работы, когда алгоритм приближается к перспективным областям, высокая мутация становится излишней и лишь вносит шум. Адаптивная мутация решает эту проблему: вероятность мутации линейно убывает от начального значения P_{start} до конечного P_{end} по мере смены поколений. Значение в поколении t вычисляется как

$$p_{\text{mut}}^{(t)} = P_{\text{start}} + \frac{t}{T-1}(P_{\text{end}} - P_{\text{start}}), \quad t = 0, 1, \dots, T-1, \quad (5)$$

где T — общее число поколений. Таким образом, ранние поколения работают с высокой мутацией, а поздние — с пониженной, что позволяет точнее уточнять найденные минимумы [1].

Вторая модификация — детектор стагнации и частичный рестарт. Ситуация, когда лучшее найденное значение целевой функции не улучшается на протяжении нескольких поколений подряд, называется стагнацией. Базовый ВВО не имеет встроенного механизма выхода из неё. Детектор стагнации отслеживает число поколений без улучшений с помощью счётчика. Как только счётчик достигает порогового значения W , алгоритм выполняет частичный рестарт: удаляет долю ρ худших особей и заменяет их новыми случайными решениями из допустимой области. Одновременно текущая вероятность мутации временно увеличивается на коэффициент $k > 1$, не превышая при этом P_{start} . После рестарта счётчик стагнации обнуляется, и линейный спад мутации возобновляется [2]. Такой механизм позволяет алгоритму выбраться из локального минимума без полного перезапуска с потерей всех найденных хороших решений.

Третья модификация — локальный поиск. Популяционные алгоритмы хорошо исследуют пространство в целом, но нередко оказываются недостаточно точными при уточнении найденных минимумов. Для устранения этого недостатка через фиксированное число поколений выбирается несколько лучших особей, для каждой из которых выполняется серия шагов локального уточнения. На каждом шаге к текущей точке добавляется случайное гауссовское возмущение с нулевым средним и масштабом, пропорциональным ширине допустимой области. Масштаб возмущения убывает с ростом номера поколения: в начале работы шаги крупнее, в конце — мельче. Если возмущённая точка даёт лучшее значение целевой функции, она принимается; в противном случае отвергается. Локальный поиск применяется только к лучшим особям, чтобы не расходовать вычислительные ресурсы на заведомо неперспективные решения [1].

Три описанных механизма дополняют друг друга: адаптивная мутация контролирует давление случайности на протяжении всего поиска, детектор стагнации обеспечивает выход из локальных ловушек, а локальный поиск улучшает точность финального результата.

1.5 Метод роя частиц и сравнение с методом биогеографической оптимизации

Метод роя частиц был предложен Дж. Кеннеди и Р. Эберхартом в 1995 году [4]. Его идея навеяна коллективным поведением стаи птиц или косяка рыб при поиске пищи. Популяция алгоритма — это рой частиц, каждая из которых соответствует потенциальному решению. Состояние частицы i задаётся её положением $x_i \in \mathbb{R}^d$ и скоростью $v_i \in \mathbb{R}^d$.

На каждой итерации t скорость и положение каждой частицы обновляются по формулам

$$v_i^{(t+1)} = \omega v_i^{(t)} + c_1 r_1 (p_{i,\text{best}}^{(t)} - x_i^{(t)}) + c_2 r_2 (g_{\text{best}}^{(t)} - x_i^{(t)}), \quad (6)$$

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)}, \quad (7)$$

где $p_{i,\text{best}}$ — лучшее положение, когда-либо посещённое частицей i ; g_{best} — лучшее положение среди всех частиц роя; ω — инерционный коэффициент; c_1 и c_2

— когнитивная и социальная составляющие соответственно; $r_1, r_2 \sim U(0,1)$ — случайные числа [5].

Инерционный коэффициент ω управляет балансом между исследованием и эксплуатацией: при большом ω частицы сохраняют текущее направление движения и исследуют пространство шире; при малом — больше притягиваются к найденным лучшим положениям. На практике ω часто линейно уменьшается в ходе работы алгоритма — от значения около 0.9 в начале до 0.4 в конце.

Несмотря на внешнее сходство — оба алгоритма работают с популяцией решений и обмениваются информацией внутри неё — механизмы этого обмена принципиально различаются.

В PSO каждая частица ориентируется на два ориентира: своё лучшее прошлое положение и глобально лучшую точку всего роя. Это делает алгоритм быстро сходящимся на унимодальных задачах: рой эффективно стягивается в область глобального минимума. Однако на сильно многоэкстремальных функциях этот же механизм становится уязвимостью: если глобально лучшая точка оказывается в окрестности локального минимума, весь рой собирается там, и выбраться оттуда без механизма перезапуска затруднительно.

В BBO обмен информацией происходит покомпонентно: каждая компонента решения может быть независимо заимствована у любого донора из популяции. Такой механизм создаёт значительно более разнообразный поток информации и препятствует полной унификации популяции. Кроме того, элитарность в BBO защищает лучшие решения, а мутация постоянно вносит случайные возмущения, поддерживая разнообразие. Всё это делает BBO более устойчивым на многоэкстремальных задачах высокой размерности, хотя и за счёт несколько более медленной сходимости на простых функциях [2].

Таким образом, PSO и BBO занимают разные ниши: PSO предпочтителен для задач с относительно простым рельефом, где важна скорость сходимости; BBO и его модификации эффективнее на задачах с выраженной мультимодальностью, где принципиально важно сохранять разнообразие популяции на протяжении всего поиска.

2 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

2.1 Требования к визуализации в реальном времени

Визуализация результатов работы алгоритмов оптимизации преследует несколько целей. Во-первых, она позволяет непосредственно наблюдать за ходом поиска и выявлять нежелательные явления: преждевременную сходимость, стагнацию, потерю разнообразия популяции. Во-вторых, она даёт возможность сравнивать несколько алгоритмов в одинаковых условиях. В-третьих, анализ поведения популяции в пространстве решений помогает понять, как именно алгоритм исследует область поиска. Перечисленные задачи порождают конкретные требования к системе визуализации, которые становятся ещё более строгими при работе на мобильном устройстве [1].

Первое требование — инкрементальность. Алгоритм оптимизации выполняется на сервере и передаёт данные мобильному клиенту итерация за итерацией. Ждать завершения всех поколений и лишь затем отображать результат неприемлемо: пользователь должен видеть, как меняется состояние популяции прямо в процессе вычислений. Это означает, что каждое обновление графика должно происходить без полной перерисовки изображения с нуля — только с добавлением новых данных к уже существующему отображению.

Второе требование — эффективность использования экранного пространства. Экран мобильного устройства существенно меньше монитора настольного компьютера. Из этого следует необходимость лаконичного представления данных: без лишних элементов, с читаемыми подписями осей и легендой, которая не перекрывает основной контент.

Третье требование — низкая вычислительная нагрузка на клиент. Построение графиков, особенно с большим числом точек, способно существенно нагрузить процессор мобильного устройства. Поэтому способы обновления и рендеринга должны быть выбраны так, чтобы частые обновления не приводили к задержкам в интерфейсе.

Исходя из перечисленных требований, можно выделить несколько типов визуального представления, актуальных для наблюдения за эволюционной оптимизацией. График сходимости показывает динамику лучшего найденного значения

целевой функции по итерациям. Карта пространства поиска отображает рельеф целевой функции и текущее положение особей популяции. Анимация процесса поиска позволяет зафиксировать всю динамику в виде воспроизводимой последовательности кадров. Каждый из этих типов решает свою задачу и будет рассмотрен в последующих подразделах.

2.2 График сходимости

График сходимости — наиболее универсальное и информативное средство анализа работы алгоритма оптимизации. По оси абсцисс откладывается номер итерации, то есть поколения, по оси ординат — лучшее значение целевой функции, найденное к данной итерации:

$$f_{\text{best}}^{(t)} = \min_{0 \leq \tau \leq t} f(x_{\text{best}}^{(\tau)}). \quad (8)$$

Такой график по определению является монотонно невозрастающим: лучшее найденное значение не может ухудшиться от итерации к итерации. По форме кривой можно судить о характере сходимости алгоритма. Резкий начальный спад с последующим выходом на плато означает быструю сходимость к некоторому значению, которое может оказаться локальным минимумом. Постепенный и равномерный спад свидетельствует о планомерном исследовании пространства. Ступенчатость кривой характерна для алгоритмов с механизмами рестарта: после каждого рестарта алгоритм находит лучшее решение, и кривая делает очередной скачок вниз [2].

С точки зрения реализации в реальном времени существуют два подхода к обновлению графика. Первый — полная перерисовка: на каждой итерации весь набор накопленных данных передается в функцию построения графика заново. Такой подход прост в реализации, но плохо масштабируется: при нескольких сотнях итераций перерисовка становится заметно медленной. Второй подход — инкрементальное добавление точки: к уже построенной кривой добавляется только новое значение, а существующие точки не пересчитываются. Это значительно эффективнее, особенно на мобильных устройствах с ограниченными вычислительными ресурсами.

При большом числе итераций на графике может оказаться слишком много точек, и кривая сольётся в сплошную полосу. В таком случае применяют скользящее окно: отображается только фиксированное число последних итераций, а по мере поступления новых данных окно сдвигается вправо. Альтернатива — прореживание: отображаются не все точки, а каждая k -я, что позволяет сохранить форму кривой при уменьшении числа отрисовываемых элементов.

Логарифмическая шкала по оси ординат применяется в случаях, когда значения целевой функции убывают на несколько порядков. Она позволяет увидеть детали поведения алгоритма на протяжении всей его работы. При линейной шкале в таких ситуациях весь интересный участок кривой сжимается у нуля и становится нечитаемым.

Для сравнения нескольких алгоритмов используется совмещённый график, где кривые сходимости разных методов отображаются на одних осях, различаясь цветом и/или типом линии. Каждый алгоритм идентифицируется в легенде. При таком представлении важно соблюдать единообразие: все методы должны запускаться на одной и той же целевой функции при одинаковом числе итераций, иначе сравнение теряет смысл.

2.3 Визуализация пространства поиска

График сходимости описывает поведение алгоритма в терминах значений целевой функции, но не даёт представления о том, где именно в пространстве решений находятся особи популяции. Для этой цели используется визуализация пространства поиска — отображение рельефа целевой функции с наложением текущего положения особей.

Прямая визуализация пространства решений возможна только при $d = 2$: в этом случае на плоскости можно построить двумерную карту, где по осям откладываются координаты решения x_1 и x_2 , а значение целевой функции $f(x_1, x_2)$ кодируется цветом. Такое представление называется тепловой картой или контурной картой. Тёмные тона соответствуют малым значениям целевой функции — то есть перспективным областям поиска, а светлые — большим [9].

Тепловая карта строится следующим образом. Допустимая область D разбивается на равномерную сетку точек. Для каждой точки сетки вычисляется значение целевой функции. Полученная матрица значений отображается как изобра-

жение с цветовой кодировкой. Поверх тепловой карты отрисовываются текущие позиции особей популяции — обычно в виде небольших маркеров. По положению облака точек на карте можно непосредственно наблюдать, в каких областях сосредоточена популяция и насколько она разнообразна.

При размерности $d > 2$ прямое отображение всего пространства решений на плоскость невозможно. В таком случае применяется проекция: для визуализации используются только первые две координаты каждой особи, остальные игнорируются. Тепловая карта при этом строится для сечения целевой функции при фиксированных нулевых значениях остальных координат. Такой подход является приближением и не отражает полной картины поиска, однако позволяет получить хотя бы частичное представление о расположении популяции даже в многомерных задачах [1].

Важным аспектом является вычислительная стоимость построения тепловой карты. Если сетка содержит $M \times M$ точек, то для отрисовки карты требуется M^2 вычислений целевой функции. При $M = 100$ это уже 10 000 вызовов, что может занять значительное время. Поэтому тепловую карту, как правило, вычисляют один раз — до запуска алгоритма — и хранят как готовое изображение. В ходе работы алгоритма обновляются только маркеры особей, а не сама карта.

2.4 Трёхмерный поверхностный график

Тепловая карта кодирует значение целевой функции цветом, что удобно для быстрой оценки рельефа, но не передаёт его объёмной структуры: относительная глубина локальных минимумов, крутизна склонов и наличие плато не всегда легко считываются по одному лишь оттенку. Альтернативным и дополняющим подходом является трёхмерный поверхностный график, в котором значение $f(x_1, x_2)$ отображается не цветом, а высотой точки в трёхмерном пространстве.

При построении такого графика допустимая область D разбивается на равномерную сетку $M \times M$, аналогично тепловой карте. Для каждой узловой точки вычисляется значение целевой функции, после чего четвёрки соседних узлов образуют четырёхугольные грани поверхности. Каждой грани присваивается цвет, определяемый средним значением f в её вершинах по той же цветовой шкале, что и для тепловой карты.

Для отображения трёхмерной сцены на двумерный экран применяется проекция. Ортографическая проекция сохраняет параллельность линий и не вносит перспективных искажений, что упрощает количественную оценку пропорций. Перспективная проекция, напротив, создаёт эффект глубины за счёт уменьшения удалённых объектов, что даёт более привычное и наглядное восприятие формы поверхности. Для задач визуализации оптимизации, где основная цель — качественное представление рельефа, а не точное измерение координат по экрану, оба варианта допустимы.

Отрисовка граней выполняется с помощью алгоритма художника: грани сортируются по удалённости от наблюдателя в порядке «от дальних к ближним» и рисуются в этом порядке. Таким образом, ближние грани перекрывают дальние, что обеспечивает корректное отображение без использования z-буфера. Для каждой грани глубина определяется как среднее значение z -координаты после поворота по четырём вершинам. Вычислительная сложность сортировки составляет $O(M^2 \log M)$, а отрисовки — $O(M^2)$, что сопоставимо со стоимостью построения тепловой карты.

Принципиальным преимуществом трёхмерного графика перед тепловой картой является возможность интерактивного вращения. Пользователь может менять точку обзора, перемещая палец по экрану. Поворот описывается двумя углами: вращением вокруг вертикальной оси φ_z и вокруг горизонтальной оси φ_x . При изменении углов координаты каждой вершины (x, y, z) пересчитываются по формулам последовательного поворота:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \varphi_z & -\sin \varphi_z \\ \sin \varphi_z & \cos \varphi_z \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}, \quad (9)$$

$$\begin{pmatrix} y'' \\ z'' \end{pmatrix} = \begin{pmatrix} \cos \varphi_x & -\sin \varphi_x \\ \sin \varphi_x & \cos \varphi_x \end{pmatrix} \begin{pmatrix} y' \\ z' \end{pmatrix}. \quad (10)$$

Экранные координаты вершины после поворота определяются как (x', y'') с учётом масштабирования и смещения к центру холста. Вращение позволяет рассмотреть поверхность под произвольным углом, что помогает обнаружить узкие «каньоны» целевой функции, как, например, у функции Букина N.6, которые плохо различимы на плоской тепловой карте.

Поверх трёхмерной поверхности наносятся те же элементы, что и на тепловую карту: маркеры текущей популяции и отметка лучшего найденного решения. Дополнительно на трёхмерном графике можно отобразить траекторию лучшего решения — последовательность точек $(x_1^{(t)}, x_2^{(t)}, f_{\text{best}}^{(t)})$, соединённых линией. Такая траектория наглядно показывает, как алгоритм продвигался по рельефу целевой функции: были ли скачки между разными локальными минимумами, насколько плавно шёл спуск, застревал ли метод на плато. В отличие от графика сходимости, который показывает только значение f от итерации, траектория на поверхности фиксирует и пространственное положение лучшего решения.

При размерности $d > 2$ для трёхмерного графика используется тот же подход проекции, что и для тепловой карты: выбираются две координатные оси для осей x_1, x_2 поверхности, а остальные координаты фиксируются на значениях лучшего найденного решения. Такое сечение проходит через окрестность оптимума и позволяет оценить локальную структуру рельефа в районе найденного минимума.

2.5 Анимированное представление динамики

Тепловая карта с наложением популяции показывает её состояние в конкретный момент времени. Однако для понимания процесса поиска важна именно динамика: как популяция смещалась от итерации к итерации, как менялось её распределение, были ли заметны рестарты или стагнации. Статичный снимок финального состояния не позволяет увидеть этот процесс.

Одним из способов зафиксировать и воспроизвести динамику является анимированный файл в Graphics Interchange Format (далее — GIF). Этот формат хранит последовательность кадров, каждый из которых представляет собой отдельное изображение. При воспроизведении кадры сменяют друг друга с заданной частотой, создавая эффект анимации [10].

GIF выбирается по ряду причин. Во-первых, он представляет собой единый самодостаточный файл, не требующий для воспроизведения каких-либо внешних плееров или кодеков: анимированный GIF поддерживается любым современным браузером и большинством мобильных приложений. Во-вторых, формат прост в генерации на стороне клиента: существуют библиотеки, позволяющие покадрово собирать GIF из набора готовых изображений. В-третьих, файл GIF удобно сохранять на устройство или передавать для последующего анализа.

Процесс генерации анимации выглядит следующим образом. В ходе работы алгоритма клиентское приложение получает от сервера данные о состоянии популяции на каждой итерации. Для каждой итерации строится кадр — тепловая карта целевой функции с наложенными маркерами особей в их текущих позициях. Кадры накапливаются в памяти. По завершении оптимизации накопленная последовательность кадров компонуется в анимированный GIF-файл.

Ключевым параметром анимации является частота смены кадров. Слишком быстрая смена кадров не позволяет разглядеть отдельные итерации; слишком медленная делает просмотр утомительным. Как правило, для алгоритмов с числом поколений порядка 100–200 достаточно задержки между кадрами в 50–100 мс, что соответствует 10–20 кадрам в секунду.

Следует учитывать ограничения формата. GIF поддерживает не более 256 цветов в одном кадре, что налагает ограничения на качество цветового градиента тепловой карты. Кроме того, при большом числе кадров и высоком разрешении размер файла может оказаться значительным. На практике для задач визуализации оптимизации эти ограничения не являются критичными: разрешение карты и число кадров выбираются исходя из компромисса между качеством изображения и размером файла [10].

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

3.1 Протоколы передачи данных и обоснование выбора WebSocket

Для организации потоковой передачи данных между сервером и мобильным клиентом необходимо выбрать подходящий протокол связи. Рассматриваемая система предъявляет к транспортному уровню несколько ключевых требований. Сервер должен отправлять данные клиенту по собственной инициативе — после каждой итерации алгоритма, не дожидаясь запроса. Соединение должно оставаться открытым на протяжении всего времени выполнения задачи. Накладные расходы на передачу каждого сообщения должны быть минимальными, поскольку сообщения о прогрессе отправляются потоком — одно за итерацию.

Рассмотрим основные варианты и их соответствие перечисленным требованиям.

Классическая модель Hypertext Transfer Protocol (далее — HTTP) с поллингом предполагает, что клиент периодически отправляет запросы на сервер с вопросом о наличии новых данных. Такой подход прост в реализации, однако неэффективен: большинство запросов возвращают пустой ответ. Кроме того, интервал поллинга определяет минимальную задержку между появлением данных на сервере и их получением клиентом [11].

Server-Sent Events (далее — SSE) — механизм HTTP, при котором сервер может отправлять клиенту поток событий по открытому соединению без повторных запросов. Это решает проблему поллинга и подходит для однонаправленной передачи данных от сервера к клиенту. Однако SSE не поддерживает двустороннюю связь: клиент не может отправлять сообщения по тому же соединению. В рассматриваемой системе клиент должен уметь не только принимать прогресс, но и передавать задачу на сервер, запрашивать статус и отправлять команду отмены. SSE для этого не подходит [12].

Протокол WebSocket обеспечивает полнодуплексное соединение между клиентом и сервером поверх TCP [11, 12]. После первоначального рукопожатия через HTTP соединение переключается на протокол WebSocket и остаётся открытым. Обе стороны могут отправлять сообщения в любой момент без дополнитель-

ных запросов. Заголовок каждого кадра WebSocket занимает всего 2–10 байт, что значительно меньше накладных расходов HTTP. Именно эти свойства — двунаправленность, постоянное соединение и низкие накладные расходы — делают WebSocket оптимальным выбором для потоковой передачи результатов оптимизации [12].

Использование WebSocket как транспорта само по себе не определяет структуру передаваемых данных. Поверх транспортного уровня необходимо определить прикладной протокол, описывающий формат и порядок обмена сообщениями. В рассматриваемой системе такой протокол строится вокруг единого конверта сообщения. Каждое сообщение содержит заголовок и полезную нагрузку. Заголовок включает версию протокола, тип сообщения, уникальный идентификатор, временную метку, а также поля для идентификации задачи и трассировки. Тип сообщения определяет его семантику: постановка задачи, подтверждение принятия, обновления очереди, прогресс выполнения, итоговый результат и служебные сообщения об ошибках.

Отдельного внимания заслуживает механизм идемпотентности. Клиент при постановке задачи указывает уникальный идентификатор запроса. Если по какой-либо причине сообщение о постановке задачи было отправлено повторно, например, из-за нестабильного соединения, сервер распознаёт дубликат и возвращает уже существующий идентификатор задачи, не создавая новую. Это защищает от случайного дублирования вычислений.

Для передачи больших массивов данных — финальной популяции или истории значений целевой функции — предусмотрен механизм разбиения на части — чанкинг. Если объём данных превышает максимально допустимый размер одного сообщения, сервер разбивает их на последовательность сообщений типа `chunk`, каждое из которых содержит порядковый номер и признак последнего фрагмента. Клиент собирает фрагменты по идентификатору потока в правильном порядке.

3.2 Серверная часть: Python

Python является де-факто стандартным языком для научных вычислений. Богатая экосистема библиотек — в первую очередь NumPy — обеспечивает эффективную работу с многомерными массивами и векторизованные вычисления. Операции над популяцией в алгоритмах BBO и PSO, например, сортировка, от-

бор доноров, вычисление целевой функции для всех особей сразу, реализуются через операции NumPy и выполняются на уровне скомпилированного кода, что значительно быстрее эквивалентных циклов на чистом Python [13].

Серверная часть системы должна обслуживать несколько клиентских соединений одновременно, не блокируя обработку одного запроса на время вычислений для другого. Python предоставляет для этого асинхронную модель программирования на основе библиотеки `asyncio`. Её ключевые понятия — сопрограммы (coroutines) и цикл событий (event loop). Сопрограмма — это функция, которая может приостановить своё выполнение в точке ожидания, например, ожидания данных по сети, и передать управление циклу событий, который тем временем обрабатывает другие задачи. Когда ожидаемое событие наступает, выполнение сопрограммы возобновляется [13]. Такой подход позволяет обслуживать множество соединений в одном потоке операционной системы без порождения нового потока на каждого клиента.

Однако сам алгоритм оптимизации является вычислительно интенсивной задачей, которая выполняется синхронно и не имеет точек ожидания. Если запустить его непосредственно в цикле событий, он заблокирует обработку всех остальных соединений на время своего выполнения. Решение состоит в запуске алгоритма в пуле потоков (thread pool): метод `loop.run_in_executor` передаёт вычислительную функцию в отдельный поток, а цикл событий продолжает работу. Прогресс передаётся из рабочего потока в цикл событий через потокобезопасную очередь с помощью `loop.call_soon_threadsafe` [13].

Важным компонентом серверной части является очередь задач. Если несколько клиентов одновременно ставят задачи на вычисление, необходимо управлять порядком их выполнения. Очередь задач решает эту задачу: каждая поступившая задача помещается в очередь с назначенным приоритетом и ожидает своей очереди. Воркер — отдельная сопрограмма — непрерывно извлекает задачи из очереди и запускает их на выполнение.

В рассматриваемой реализации очередь построена на основе `asyncio.PriorityQueue`. Каждый элемент очереди хранит три поля: числовой приоритет, порядковый номер постановки для разрешения коллизий при равных приоритетах и идентификатор задачи. По идентификатору воркер извлекает полный объект задачи из реестра. Поддерживаются три уровня

приоритета: высокий, обычный и низкий. Задача с более высоким приоритетом будет обработана раньше, чем задача с низким, даже если та была поставлена первой [13].

Жизненный цикл задачи в очереди включает несколько состояний. После постановки задача переходит в состояние ожидания (QUEUED). В этом состоянии клиент может запрашивать свою позицию в очереди и расчётное время ожидания. Когда воркер берёт задачу в работу, она переходит в состояние выполнения (RUNNING), и сервер начинает отправлять клиенту сообщения о прогрессе. По завершении задача переходит в одно из терминальных состояний: успешное завершение (SUCCEEDED), ошибка (FAILED), отмена пользователем (CANCELLED) или превышение таймаута (TIMEOUT). Терминальное состояние необратимо: задача остаётся в реестре, и клиент может в любой момент запросить её статус или результат.

3.3 Формат обмена данными: JSON

Для сериализации сообщений между сервером и клиентом используется формат JavaScript Object Notation (далее — JSON). JSON представляет данные в виде текста с использованием структур двух типов: объектов в виде пар ключ — значение и массивов. Формат поддерживается нативно практически во всех современных языках программирования и платформах, включая Python и Dart [14, 15].

Главные достоинства JSON применительно к данной задаче — это читаемость и простота отладки. Сообщения протокола можно просматривать в текстовом виде без специальных инструментов, что существенно упрощает разработку и тестирование. Схема сообщений при этом остаётся гибкой: добавление новых полей в структуру не нарушает обратной совместимости со старыми версиями клиента.

Существуют более компактные бинарные форматы сериализации — Concise Binary Object Representation и Google Protocol Buffers. Они обеспечивают меньший размер сообщений и более высокую скорость разбора за счёт двоичного кодирования данных [16]. Однако выигрыш в производительности, который они дают, не является определяющим фактором. Сообщения о прогрессе содержат ограниченный набор числовых полей, и их JSON-представление достаточно компактно.

Накладные расходы на сериализацию пренебрежимо малы по сравнению с временем самих вычислений.

Отдельного рассмотрения заслуживает передача числовых массивов — финальной популяции и истории сходимости. Популяция размером $N \times d$ при $N = 50$ и $d = 10$ содержит 500 чисел с плавающей точкой. В JSON-представлении каждое число занимает в среднем около 10–15 символов, то есть весь массив — около 5–7 КБ. Это приемлемо для однократной передачи по завершении задачи. Для передачи в потоке прогресса по итерациям массив популяции, как правило, не включается: клиент получает только лучшее значение и лучшую позицию на текущей итерации, что на порядок меньше по объёму. Полный массив передаётся однократно в итоговом сообщении с результатом, при необходимости — по механизму чанкинга.

3.4 Клиентская часть: Dart и Flutter

Dart — современный язык программирования, разработанный компанией Google. Он поддерживает как объектно-ориентированную, так и функциональную парадигмы и обладает строгой статической типизацией, что помогает обнаруживать ошибки на этапе компиляции [17]. Ключевой особенностью Dart применительно к данной задаче является встроенная поддержка асинхронного программирования на основе двух абстракций: `Future` и `Stream`. `Future` представляет одиночный результат асинхронной операции, который будет доступен в будущем. Это аналог `Promise` в JavaScript. `Stream` — это последовательность асинхронных событий: именно эта абстракция естественным образом соответствует потоку сообщений от сервера через `WebSocket`. Входящие сообщения появляются в `Stream` по мере их прихода, и приложение подписывается на них с помощью оператора `listen` или оператора `await for` в асинхронном цикле [17].

Flutter — кроссплатформенный фреймворк для разработки пользовательских интерфейсов, основанный на Dart [18]. Приложение на Flutter компилируется в нативный код для целевой платформы, что обеспечивает высокую производительность. Фреймворк использует собственный движок рендеринга и не зависит от нативных виджетов операционной системы. Интерфейс описывается деревом виджетов: каждый элемент UI представлен объектом с описанием своего содержимого и оформления. При изменении состояния, например, при получении новых

данных от сервера, Flutter перестраивает затронутую часть дерева виджетов и обновляет только изменившиеся элементы экрана.

Для управления состоянием при потоковом обновлении данных в Flutter используется несколько подходов. Простейший — виджет `StreamBuilder`, который подписывается на `Stream` и автоматически перестраивает своё поддерево при каждом новом событии. Это удобно для графиков сходимости: `StreamBuilder` получает очередную точку данных и передаёт обновлённый список точек в компонент построения графика [18].

Построение графиков в Flutter реализуется с помощью специализированных пакетов. Пакет `fl_chart` предоставляет компоненты для построения линейных и других типов графиков с поддержкой анимированного обновления. Он позволяет задать набор точек данных и описать внешний вид осей, сетки и линий. Для тепловой карты пространства поиска используется компонент `CustomPainter` — низкоуровневый API рисования, который предоставляет доступ к холсту и позволяет отрисовывать произвольные графические примитивы: прямоугольники, точки, окружности. Тепловая карта реализуется как матрица закрашенных прямоугольников, цвет которых определяется значением целевой функции. Тот же механизм `CustomPainter` применим для построения трёхмерного поверхностного графика: грани поверхности рисуются как многоугольники с заливкой и обводкой, а сортировка по глубине и пересчёт координат при повороте выполняются средствами языка Dart.

Работа с `WebSocket` в Dart реализуется через класс `WebSocket` из стандартной библиотеки `dart:io` для мобильных платформ или `WebSocketChannel` из пакета `web_socket_channel`. После установки соединения входящие сообщения доступны через `Stream`, а для отправки используется метод `sink.add`. Разбор входящих JSON-сообщений выполняется функцией `jsonDecode` из стандартной библиотеки `dart:convert` [15]. Таким образом, весь цикл обработки сообщений — приём, разбор, обновление состояния, перерисовка UI — является полностью асинхронным и не блокирует поток интерфейса [17, 18].

ЗАКЛЮЧЕНИЕ

В ходе настоящей научно-исследовательской работы проведено теоретическое исследование по трём направлениям, необходимым для разработки программно-аппаратного комплекса поддержки мобильного приложения визуализации результатов работы модификаций эволюционного алгоритма оптимизации.

В первом разделе рассмотрены теоретические основы алгоритма биогеографической оптимизации. Изучена модель миграции Саймона и механизм частичной иммиграции, лежащий в основе классического ВВО. Проанализированы синусоидальная и смешанная модели миграционных кривых как способы изменить баланс между исследованием пространства решений и уточнением найденных минимумов. Рассмотрены три механизма модификации базового алгоритма: адаптивная мутация с линейным убыванием вероятности, детектор стагнации с частичным рестартом и локальный поиск гауссовскими возмущениями. Проведён сравнительный анализ ВВО и PSO: показано, что алгоритмы различаются по механизму обмена информацией в популяции и занимают разные ниши применения — PSO эффективнее на задачах с простым рельефом, тогда как модифицированные варианты ВВО предпочтительнее на сильно мультимодальных функциях высокой размерности.

Во втором разделе исследованы подходы к потоковой визуализации результатов оптимизации на мобильном устройстве. Сформулированы требования к визуализации в реальном времени: инкрементальность обновлений, эффективное использование экранного пространства и низкая вычислительная нагрузка на клиент. Проанализированы четыре типа представлений. График сходимости позволяет наблюдать за динамикой лучшего найденного значения целевой функции; рассмотрены подходы к его инкрементальному обновлению, применение логарифмической шкалы и совмещение кривых нескольких алгоритмов для сравнения. Тепловая карта пространства поиска с наложением популяции даёт возможность визуально оценить распределение особей относительно рельефа целевой функции; для задач с размерностью выше двух обоснован метод проекции на выбранные координаты. Трёхмерный поверхностный график дополняет тепловую карту, отображая значение целевой функции как высоту точки поверхности; описаны алгоритм художника для корректной отрисовки граней, формулы поворота для

интерактивного вращения и наложение траектории лучшего решения. Анимированный GIF выбран как способ зафиксировать всю динамику процесса поиска в виде единого воспроизводимого файла; обоснованы его преимущества перед статичным снимком и рассмотрены ограничения формата.

В третьем разделе исследован программный стек для реализации системы. Проведён сравнительный анализ протоколов связи — HTTP polling, SSE и WebSocket — и обосновано, что WebSocket наиболее соответствует требованиям двунаправленной потоковой передачи данных. Описаны принципы построения прикладного протокола поверх WebSocket: структура конверта сообщения, типы сообщений, механизм идемпотентности и чанкинг для передачи больших массивов. На стороне сервера рассмотрена асинхронная модель Python на основе `asyncio`, обоснован запуск вычислительно интенсивного алгоритма в пуле потоков, а также проанализированы принципы организации самописной очереди задач с приоритизацией и управлением жизненным циклом. Рассмотрен формат JSON как средство сериализации сообщений, его достоинства для прототипирования и оценён объём передаваемых числовых данных. На стороне клиента исследованы язык Dart с его асинхронной моделью на основе `Future` и `Stream`, а также фреймворк Flutter с возможностями `StreamBuilder`, `fl_chart` и `CustomPainter` для реализации потоковой визуализации.

Полученные результаты формируют теоретическую основу для последующей практической разработки: выбора архитектурных решений, реализации серверной и клиентской частей комплекса и их интеграции в единую систему.

СПИСОК ЛИТЕРАТУРЫ

1. Карпенко А. П. *Современные алгоритмы поисковой оптимизации*: учеб. пос. – М.: Изд-во МГТУ им. Н.Э. Баумана, 2014. – 446 с.
2. Родзин С. И., Скобцов Ю. А., Эль-Хатиб С. А. *Биоэвристики: теория, алгоритмы и приложения*. – Чебоксары: Изд-во «Среда», 2019. – 224 с.
3. Гладков Л. А., Курейчик В. В., Курейчик В. М. *Генетические алгоритмы*. – М.: Физматлит, 2010. – 368 с.
4. Kennedy J., Eberhart R. Particle swarm optimization // *Proceedings of ICNN'95 – International Conference on Neural Networks*. – 1995. – Vol. 4. – P. 1942–1948.
5. Карпенко А. П., Щербакова Н. О., Буланов В. А. Гибридный алгоритм глобальной оптимизации на основе алгоритмов искусственной иммунной системы и метода роя частиц // *Наука и образование (электрон. журн. МГТУ им. Н.Э. Баумана)*. – 2014. – №3. – С. 255–271.
6. Саймон Д. *Алгоритмы эволюционной оптимизации* / пер. с англ. А. В. Логунова. – М.: ДМК Пресс, 2020. – 1002 с.
7. Карпенко А. П., Синяговская О. А. Глобальная оптимизация методом биогеографии // *Наука и образование (электрон. журн. МГТУ им. Н.Э. Баумана)*. – 2013. – №10. – С. 373–398.
8. Карпенко А. П. Эволюционные операторы популяционных алгоритмов глобальной оптимизации. Опыт систематизации // *Математика и математическое моделирование*. – 2018. – №1. – С. 59–89.
9. Новикова Е. С., Бекенева Я. А., Бестужев М. П. Методы визуального анализа для мониторинга данных от сенсорных сетей // *Известия СПбГЭТУ «ЛЭТИ»*. – 2020. – №8-9. – С. 52–60.
10. GIF89a Graphics Interchange Format Version 89a // *CompuServe Incorporated*. – 1990. – 34 p.
11. Олифер В. Г., Олифер Н. А. *Компьютерные сети. Принципы, технологии, протоколы*. – 5-е изд. – СПб.: Питер, 2016. – 992 с.
12. Федулов А. А. Проектирование архитектуры протокола клиент-серверного взаимодействия web-приложений на основе websocket // *Программные системы и вычислительные методы*. – 2025. – №3. – С. 20–30.

13. Яворски М., Зиаде Т. *Python. Лучшие практики и инструменты* – СПб.: Питер, 2024. – 592 с.
14. *json — JSON encoder and decoder // Python 3 documentation*: [сайт]. – URL: <https://docs.python.org/3/library/json.html> (дата обращения: 09.04.2026).
15. *dart:convert // Dart documentation*: [сайт]. – URL: <https://dart.dev/libraries/dart-convert> (дата обращения: 09.04.2026).
16. Шульман В. Д., Шабанов В. В., Сухов П. А., Чунихин А. О. Сравнительный анализ форматов сериализации и передачи данных JSON, XML, CBOR и GPB // *StudNet*. – 2021. – Т. 4, №7. – С. 1686–1696.
17. Рябоконь О. С., Кукарцев В. В. Новый язык структурного веб-программирования Dart // *Актуальные проблемы авиации и космонавтики*. – 2013. – Т. 1, №9. – С. 436–437.
18. Камышев А. О., Зотин А. Г. Особенности использования фреймворка Flutter при разработке кроссплатформенных приложений // *Актуальные проблемы авиации и космонавтики*: сб. материалов VI Междунар. науч.-практ. конф. – Красноярск: СибГУ им. М. Ф. Решетнёва, 2020. – Т. 2. – С. 138–140.